

模块五：评测分析与展示能力 (Analysis & Presentation)

该接口用于获取评测任务的可视化分析数据。支持生成单次执行的聚合评分（用于雷达图）、执行明细列表（用于下钻分析）以及跨执行批次的横向对比数据。

1. 执行明细 (Execution Case) 关键字段说明

参数名	类型	说明
evaluation_results	JSON	存储各指标的具体得分，如 <code>{"metric_1": 0.8, "metric_2": 1.0}</code>
execution_details	JSON	包含该样本的性能数据： <code>latency</code> , <code>token_usage</code> , <code>steps</code>
status	String	样本运行状态： <code>PASSED</code> , <code>FAILED</code> , <code>ERROR</code>

2. 评估方法 (Evaluation Method) 关键字段说明

参数名	类型	说明
dimension	String	用于前端分类展示：效果、安全、性能
output_type	String	决定前端渲染方式：分数（进度条）、布尔值（勾叉）

evaluation_methods（评估方法定义）

- 目标：把评估逻辑抽象为可配置对象，支持内置与自定义方法。
- 字段原因：
 - id/name/description：方法识别与说明。

- category/dimension/mode/output_type：方法分类体系，支撑编排。
- is_builtin：区分系统内置与用户扩展。
- parameters（JSON）：参数定义模板。
- implementation（JSON）：自定义方法实现信息（脚本/API/Prompt）。
- usage_count：评估方法使用统计。
- created_at/updated_at/last_used_at：方法生命周期管理。

基础配置

- 1. 响应格式：统一返回 `Result<V0>` 对象。
- 2. Host: `localhost:8080`
- 3. 路径前缀: `/api/analysis`

接口列表

接口名称	接口描述	Http方法	URL
getExecutionSummary	获取单次执行的聚合汇总数据	GET	<code>/api/analysis/executions/{id}/summary</code>
getCaseAnalysisList	分页获取样本执行明细（含评分）	GET	<code>/api/analysis/executions/{id}/cases</code>
getExecutionComparison	跨任务/跨批次对比分析	GET	<code>/api/analysis/compare</code>
getMetricDefinitions	获取本次执行涉及的指标定义	GET	<code>/api/analysis/executions/{id}/metrics</code>

关键接口请求示例

1. 获取执行汇总 (getExecutionSummary)

GET `localhost:8080/api/analysis/executions/20240408001/summary`

说明：用于展示执行概览。后端会根据 `execution_cases` 中的 `evaluation_results` 计算各项指标的平均分，并统计总耗时和 Token。

Response:

代码块

```
1  {
2      "code": 200,
3      "data": {
4          "execution_id": 20240408001,
5          "overall_score": 0.88,
6          "pass_rate": 0.92,
7          "total_latency": 125.5,
8          "avg_latency_per_case": 12.5,
9          "total_tokens": 4500,
10         "dimension_scores": [
11             { "dimension": "EFFECTIVENESS", "score": 0.85 },
12             { "dimension": "PERFORMANCE", "score": 0.90 },
13             { "dimension": "SAFETY", "score": 1.0 }
14         ]
15     },
16     "msg": null
17 }
```

2. 获取样本执行明细 (getCaseAnalysisList)

GET `localhost:8080/api/analysis/executions/20240408001/cases?status=FAILED&page=1`

说明：用于“深度诊断”。用户可以筛选出失败的样本，查看 Agent 的输出与标准答案的差异。

Response:

代码块

```
1  {
2      "code": 200,
3      "data": {
4          "list": [
5              {
6                  "case_id": "item_005",
7                  "status": "FAILED",
8                  "input_data": { "content": "..."} },
9              ...
10         ]
11     }
12 }
```

```

9         "output_data": { "verdict": "True" },
10        "expected_output": { "verdict": "False" },
11        "evaluation_results": {
12            "ragas_faithfulness": 0.2,
13            "is_correct": 0
14        },
15        "error_message": "判定结果与标准答案不符"
16    }
17 ],
18     "total": 5
19 }
20 }

```

3. 跨任务对比分析 (getExecutionComparison)

GET <localhost:8080/api/analysis/compare?ids=20240408001,20240409005>

说明：核心扩展能力。对比两个不同版本的 Agent 在同一个数据集上的表现。前端通常用双雷达图或对比表格展示。

Response:

代码块

```

1  {
2      "code": 200,
3      "data": {
4          "metrics": ["准确率", "响应耗时", "Token成本", "忠实度"],
5          "series": [
6              {
7                  "execution_name": "Sonnet-V1-ReAct",
8                  "scores": [0.85, 12.5, 450, 0.88]
9              },
10             {
11                 "execution_name": "Sonnet-V2-Reflexion",
12                 "scores": [0.92, 18.2, 680, 0.95]
13             }
14         ]
15     }
16 }

```

业务逻辑解释与前端建议

1. 数据的“深度诊断”路径

- 用户在 **任务列表** 看到分数不理想。
- 点击进入 **Summary 视图**（调用 `getExecutionSummary`），通过雷达图发现“忠实度（Faithfulness）”分数很低。
- 点击雷达图中的指标，跳转到 **Case 列表**（调用 `getCaseAnalysisList`），过滤出该指标得分低的样本。
- 点击具体样本，查看 **Trace 日志**（模块四接口），发现 Agent 搜索到的网页内容是对的，但推理逻辑（Thinking）出现了偏差。

2. 字段原因在分析中的体现

- `execution_details` (JSON): 这里存储的 `response_time` 和 `token_usage` 是计算“性能维度”得分的原材料。
- `evaluation_results` (JSON): 这里存储了多个维度的评分（如 Ragas 算出的 3 个子项）。这种设计允许前端在界面上自由切换查看“基础准确率”还是“LLM 深度评价”。

3. 可视化建议

- **单任务分析**: 建议使用 **雷达图**（展示维度得分）和 **散点图**（展示每条样本的耗时分布）。
- **对比分析**: 建议使用 **分组柱状图**。例如，对比 V1 和 V2 在 `FACTUAL_ERROR`、`LOGICAL_FALLACY` 等不同错误类型上的召回率。

4. 统计性能优化

由于 `execution_cases` 可能会有成千上万条记录，后端在实现 `getExecutionSummary` 时，建议在任务完成时将聚合结果存入 `eval_tasks` 的 `result_summary` 字段，而不是每次接口请求都实时计算全表，从而保证前端加载极其流畅。

最后集成 Ragas: 在任务完成后调用 Ragas 脚本更新评分。

这样你就能快速看到一个** “输入一篇文章 -> Agent 开始思考并查网页 -> 网页显示思考过程” **的完整 Demo 了。